

Reflection:

One meaningful weakness from a prior iteration

The most meaningful weakness from a prior iteration that we had corrected would be our branching workflow. More specifically, we were inexperienced with our understanding and our overall branching strategy as a team. This definitely led to some serious issues when it came time to merging and overall developmental issues. Such as, issues with our branches being too many commits behind from the branch we would merge to, leading to prudent and inadvertent technical debt. This would manifest its way by how long it would take us to resolve all merge conflicts between different branches and the mental overhead of juggling the context of the different branches when it came down to choosing which implementation should be considered. Ultimately because of how much time spent out of sync, and the different implementations of methods/classes caused us to scrap a branch's work entirely. Forcing us to take on the implementation of one and reimplementing the other scrapped branch on the one we compromised on.

Concrete corrective actions taken

The first thing we did was to address this issue during our retrospective after iteration 1. We had identified the issues we have stated above and came up with our solution. We researched different types of branching strategies and decided that gitflow strategy was what we were going to follow. Gitflow has 2 long lived branches, the main and the development branch, followed by our other branches for features, technical debt, testing, and anything else. We would leave the main branch alone and only merge into that branch once everything in the development branch was where we wanted to be in our project. The other branches were for anything else and the rule was to merge your branches onto the development branch. Another thing we emphasized on was the time spent in those branches and the amount of work being done. Since, previously we spent too much time in one branch which became a monolith for one big feature, this caused major issues. Thus, we pivoted to making sure that the other branches, aside from the main and development branch, were to hold granular issues or implementation and to make sure we were committing often and merging into the development branch. Reducing the risk and the severity of the issues that would potentially arise when it came time to merging.

Measurable evidence of improvement

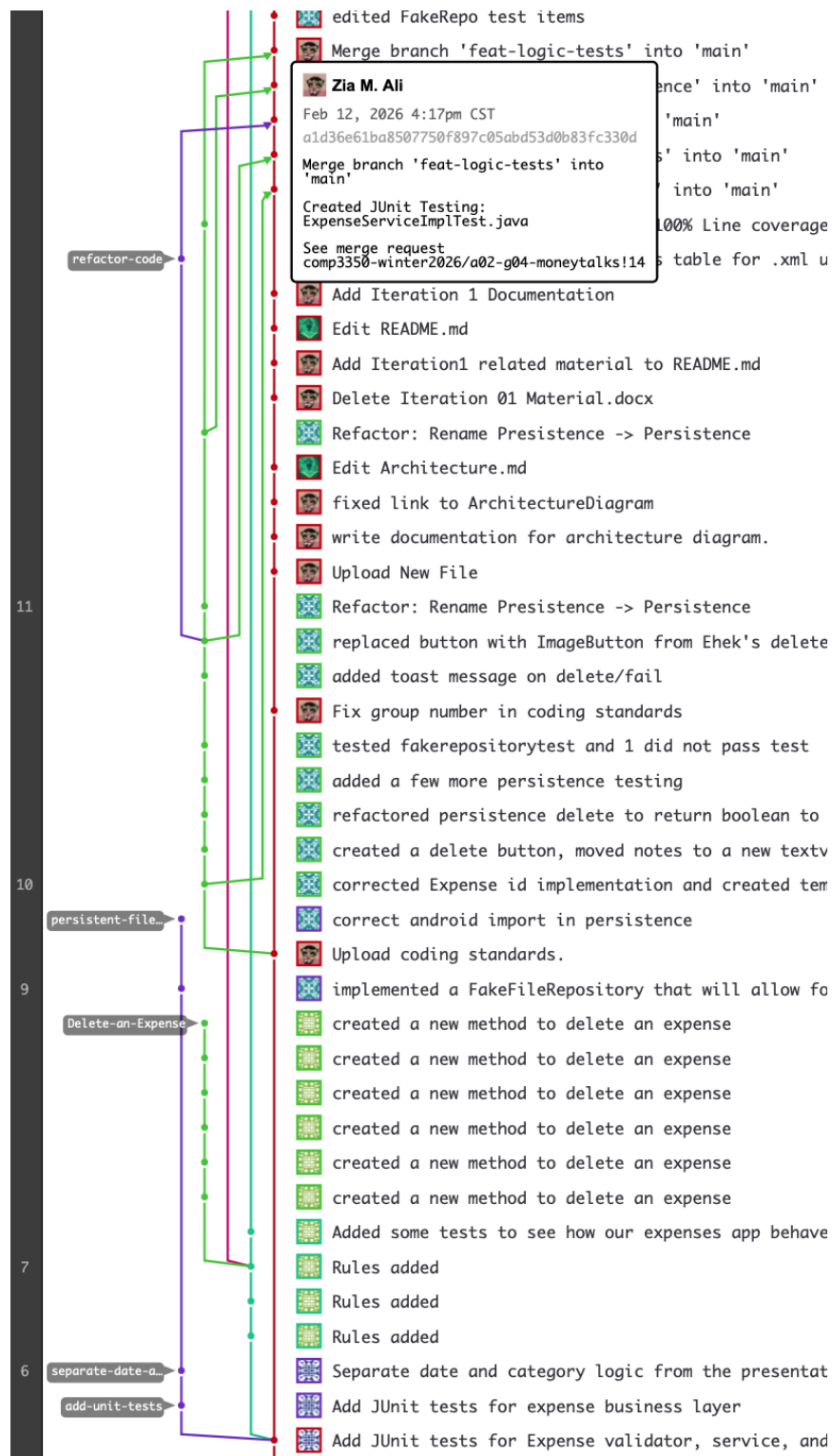


Figure 1: Before Gitflow Strategy

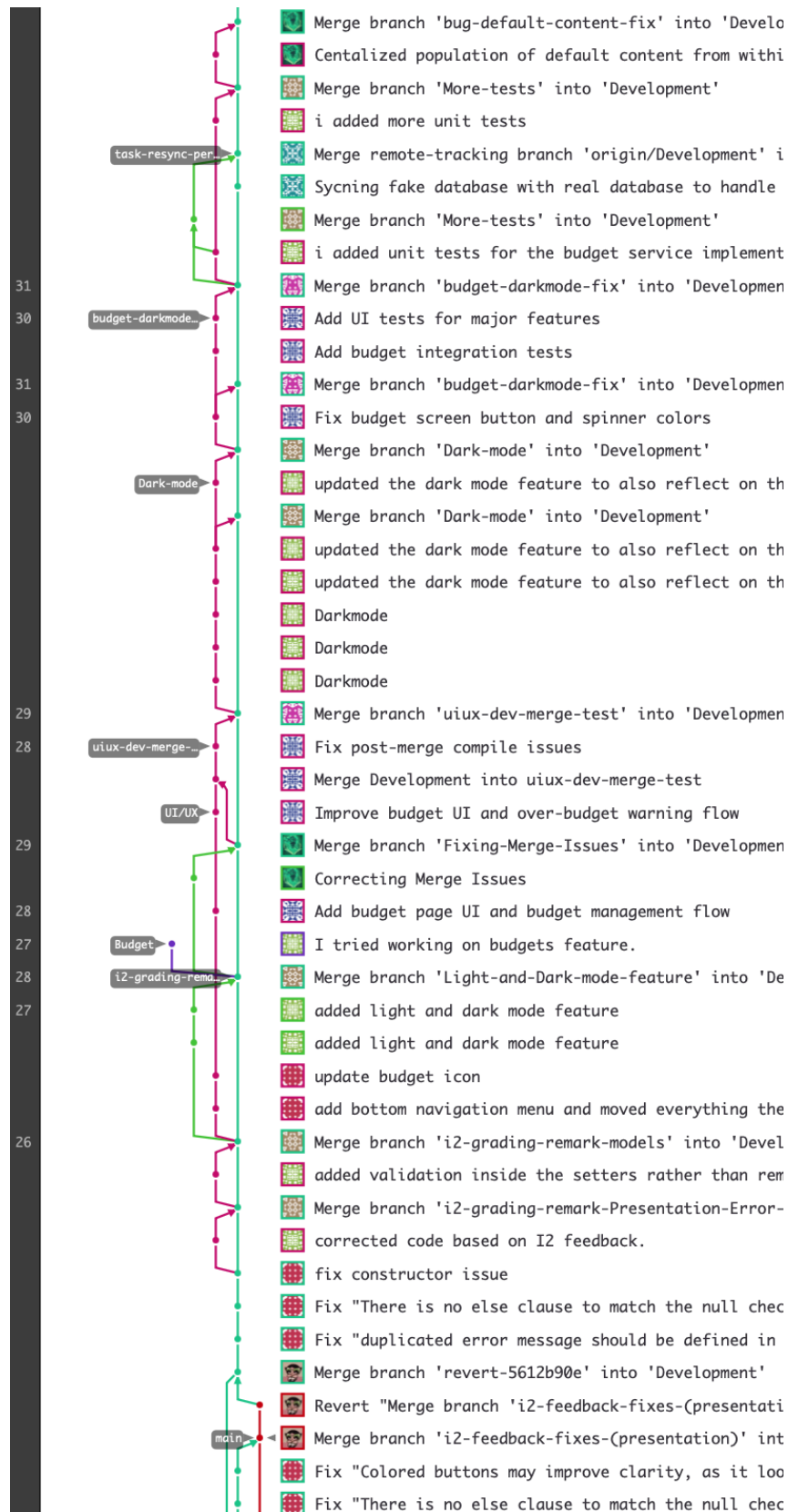


Figure 2: After Gitflow Strategy

We have our before (figure 1) and after (figure 2) of implementing Gitflow strategy. Here we can see our branching strategy or lack thereof in effect, although we thought we were doing an okay job with using different branches. We lacked committing often and merging into our core branch, ending that branch, and creating a new branch for the next thing. As seen in figure 1, some branches were initially branched off on Feb. 6th, another on the 7th, another on the 9th, and another on the 11th. This caused too much time out of sync from one another's implementation that once it came time to merge, there were merge conflicts that had to be resolved.

This is one thing that we rectified using our new branch workflow strategy. As we can see, there is much less time spent in general, and the turn-around time for branch merge has been reduced from multiple days that span to almost a week, to pretty much the same day or the next day. In turn, we are merging often with the development branch and branching off into branches that handle more fine grain issues or workload which reduces our potential risk and severity of merge conflicts down the road, actively combating technical debt.

Velocity Chart: across all three iterations

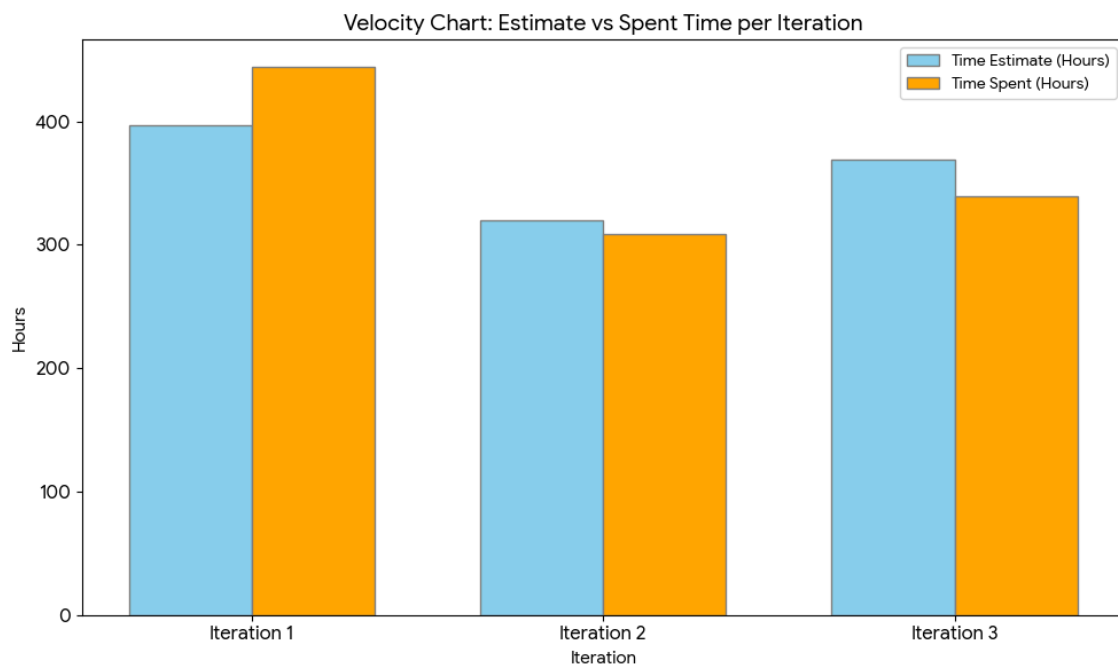


Figure 3: Velocity Chart

Above is the velocity chart for our project across all three iteration milestones. In iteration 1, we can see that our estimates were inaccurate and this caused us to overspend with the actual amount of time for the issues we have created. In iteration 2 we tried to adjust our numbers to our version of estimates and tracked our time spent. In iteration 3, these numbers jump up a bit more because of the work that we had to do to complete all that we needed done. One of the biggest issues we have here is that our estimate and spent time, as someone could say, is somewhat overexaggerated. If we had more iterations, this would

be one of the key talking points that I would have brought up, making sure that our numbers and expectations properly reflect reality.